

Nome: Pedro Santos Lacerda

Atenção: As questões que solicitarem o projeto de um algoritmo deverão conter em sua resposta (1) o pseudocódigo do algoritmo, ou (2) uma descrição textual sem ambiguidade com os passos necessários para implementá-lo.

Questão 1 (2 pontos)

Determine uma função $f(n)$ para o código a seguir tal que $T(n) = \theta(f(n))$. Explique como a solução foi encontrada.

semelhante ao merge sort!

HashTable t;

```
void f(int v[], int startInx, int endInx) {
```

```
    if (startInx < endInx - 1) {
```

```
        int midInx = (startInx + endInx) / 2;
```

```
        f(v, startInx, midInx);
```

```
        f(v, midInx, endInx);
```

```
        for (int i=startInx; i < endInx; i++) {
```

```
            t[v[i]]++;
```

```
        }
```

```
        int r[endInx - startInx];
```

```
        int aInx = startInx;
```

```
        int bInx = midInx;
```

```
        int rInx = 0;
```

```
        while (aInx < midInx && bInx < endInx) {
```

```
            r[rInx] = v[aInx] <= v[bInx] ? v[aInx++] : v[bInx++];
```

```
            r[rInx++] += t[r[rInx]];
```

```
        }
```

```
        while (aInx < midInx) {
```

```
            r[rInx] = v[aInx++];
```

```
            r[rInx++] += t[r[rInx]];
```

```
        }
```

```
        while (bInx < endInx) {
```

```
            r[rInx] = v[bInx++];
```

```
            r[rInx++] += t[r[rInx]];
```

```
        }
```

*iteração
n → O(n)*

se for 1, faz NADA :)

→ se tamanho > 1

*→ método
→ recursivamente → $\frac{n}{2}$*

*iteração em n → O(n)
faz contagem de frequências*

→ tamanho n, vetor auxiliar

→ índices para andar no esquerdo e no direito

→ seleciona o menor entre a e b

e avança

(a → primeiro método)

(b → segundo método)

terminou b, faz a

(não faz mais nada)

terminou a, faz b

```
(for (aInx = startInx; aInx < endInx; ++aInx) {  
    v[aInx] = r[aInx - startInx];  
})
```

item pelo lado de novo.
→ insere o que está em r no array principal

} if
} void

Questão 2 (1 ponto)

A seguir são apresentados cenários que exigem a escolha de um algoritmo de ordenação. Dentre os algoritmos apresentados em aula, proponha um adequado para cada cenário, buscando a maior eficiência de tempo e espaço. Explique a sua escolha.

- Uma estação meteorológica precisa manter continuamente ordenados os n maiores valores de temperatura em um sistema com memória limitada. Os dados chegam em tempo real, incluindo temperaturas com muitas casas decimais em uma ampla faixa de valores, e o conjunto é atualizado frequentemente. É essencial que o algoritmo apresente complexidade $O(n \log n)$ no pior cenário.
- Uma empresa de logística precisa ordenar 10 milhões de códigos de rastreio de encomendas. Cada código consiste em 12 dígitos numéricos (por exemplo, 202407260134). A empresa precisa ordenar esses códigos frequentemente para otimizar as rotas de entrega e o processo de triagem nos centros de distribuição. O sistema tem memória suficiente para processar todos os códigos de uma vez.

Questão 3 (2 pontos)

Uma empresa de análise de redes sociais, que possui uma base de dados com milhões de usuários, está desenvolvendo um algoritmo para identificar influenciadores emergentes. Além de informações cadastrais, em cada usuário é armazenado o seu número de seguidores e uma pontuação representando o seu índice de engajamento, calculado com base em uma fórmula envolvendo likes, comentários e compartilhamentos. Um influenciador emergente é um usuário que está no top 10% em termos de engajamento, porém não está no top 10% em número de seguidores. Projete um algoritmo eficiente que, dada uma sequência A com n tuplas $\langle id_usuário, seguidores, engajamento \rangle$, identifique todos os influenciadores emergentes. O algoritmo deverá apresentar uma complexidade $O(n)$.

Caso seja necessário, é permitido nessa questão fazer referência a algoritmos apresentados em aula para compor a solução.

Questão 4 (5 pontos)

Uma sequência A de tamanho n contém inteiros positivos e negativos.

- Projete um algoritmo $O(n^3)$ capaz de determinar os índices i e j , sendo $i < j$ tal que a soma de $A[i] + \dots + A[j]$ é máxima.
- Otimize a solução do item anterior produzindo um algoritmo $O(n^2)$.

- c) Avalie se é possível produzir um algoritmo $O(n)$ para esse problema. Caso seja possível, apresente o algoritmo, caso contrário prove que é impossível.

Nessa questão não é permitido apenas fazer referência a algoritmos apresentados em aula para compor a solução. Tais algoritmos podem ser utilizados, no entanto devem ser descritos explicitamente.

① Podemos observar logo de cara que o algoritmo faz uso de uma hash table. Como foi estudado que, no caso médio, hash tables realizam operações de inserção e busca em tempo constante $O(1)$, esse tempo será considerado aqui.

complexidade $O(1)$
O algoritmo antes de tudo verifica se a entrada tem tamanho 1, se tiver, modo é feito. Para entradas maiores, o que é feito é:

- Partir o vetor em dois na metade (sempre na metade) e fazer uma chamada recursiva tanto na esquerda quanto na direita (2 chamadas com $\frac{n}{2}$ em cada)
- Iterar pelo vetor (após as chamadas recursivas) e fazer uma contagem de frequências dos elementos usando uma hash table. Como a hash table apresenta modificações em $O(1)$, essa etapa tem complexidade $O(n)$ sempre (faz $O(1)$ n vezes)
- criar um vetor auxiliar de tamanho n , começar a andar na metade esquerda e no método direto do array, e inserir elementos procurados no vetor auxiliar. São 3 whiles, sendo que o primeiro termina quando o algoritmo termina de andar em uma das metades do array, e algum dos outros while termina de andar no outro. Como as operações em hash table são constantes, aqui temos n iterações fazendo operações $O(1)$, então essa etapa tem complexidade $O(n)$

semelhante a etapa de merge do mergesort

- Por último o algoritmo copia todos os elementos do vetor auxiliar para o vetor de entrada, com custo $O(n)$, já que faz operações constantes n vezes

No geral, esse algoritmo é bem semelhante ao Merge Sort, e sua recorrência, baseado nas análises dadas, é

$$T(n) = \begin{cases} O(1) & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + O(n) & \text{se } n > 1 \end{cases}$$

Como $n^{\log_2 2} = n$, sua recorrência se encaixa no segundo caso do Teorema Mestre e então $T(n) = O(n \log n)$ ✓ 2.0 ($f(n) = n^{\log_2 2}$)

(2*) a) Como a estação tem memória limitada, podemos descartar algoritmos que tenham complexidade de espaço $O(n)$, como o merge sort. Também podemos descartar o quicksort, já que sua complexidade em pior caso pode ser $O(n^2)$.

Logo, o algoritmo mais adequado é o Heap Sort. Ele é realizado in-place no array e não necessita de ponteiros para construir a heap.

Mais especificamente, o que pode ser feito é:

- Construir inicialmente um max-heap no vetor de temperaturas (max-heapify ou primeiro método)
- Excluir o heap sort nos n primeiros elementos e invertê-los

Uma das vantagens do heap sort, além de ser $O(n \log n)$ e as operações in-place, é que ele permite ordenar apenas os n primeiros elementos, o que é vantajoso caso a empresa tenha mais de n temperaturas registradas

PODERIA
ELABORAR
MAIS

✓ 0.4

↳ Como o sistema tem memória suficiente, não é necessário persistir em um algoritmo que faça uso de disco. Como o número de dígitos em um código postal é fixo (12) e a quantidade de dígitos também (10), é possível usar radix sort para fazer a ordenação. Como sabemos, a complexidade desse algoritmo é

$O(w(n+k))$, sendo w o nº de dígitos e k o tamanho do alfabeto deles.

Como 10 milhões é maior que 12 e 10 em vários ordens de magnitude, o algoritmo terá no máximo complexidade $O(n)$, o que representa uma vantagem para a empresa, que poderá aumentar a base de códigos de rastreio sem ter um aumento não-linear nos tempos de ordenação.

✓ 0.5



Esse algoritmo principal, inclui o algoritmo mediana das medianas, porém modificado.

Como cada elemento da lista é uma tupla, é preciso que as comparações na mediana das medianas sejam feitas em algum dos índices da tupla (ou seja, no lugar de fazer

③

$A[i] < A[j]$, fazer $A[i][index] < A[j][index]$). O index será passado ~~na~~ para o algoritmo,

que irá passar para todas as suas instâncias que serem comparáveis.

$MOM(v, start, end, search, index)$ ← representação da versão modificada

Além disso, inclui o algoritmo de particionamento visto em aula, mas com uma modificação: ele irá procurar por um elemento, que será usado para particionar.

$PARTICIONAPORELEMENTO(v, start, end, elemento, index)$

- para $i = start$ até end :

- se $v[i] == elemento$:

- swap(v, i, end)

- break;

- PARTICIONA($v, start, end, index$) - $O(n)$

$\left. \begin{array}{l} O(n) \\ O(n) \end{array} \right\} \text{ total } O(n)$

O algoritmo particiona é o visto em aula, que particiona pelo último elemento.

Ele foi modificado assim como o MOM para fazer comparações pela tupla

Por último, inclui hash tables, e o conjunto de chaves será o conjunto de IDs de memórias (ou seja, uma chave representa um endereço)

ENCONTRA EMERGENTES (A, n)

define hashtable H

define emergentes $[n/10]$

define topengajamento = MOM($A, 0, n, \frac{9n}{10}, 3$) $\rightarrow$ paga o TOP 10% engajamento

define topseguidores = MOM($A, 0, n, \frac{9n}{10}, 2$) $\rightarrow$ " " " " seguidores

PARTICIONA POR ELEMENTO($A, 0, n, \text{topseguidores}, 2$) $\rightarrow$ todos depois do índice $\frac{9n}{10}$ são TOP 10% seguidores

- para $i = \frac{9n}{10}$ até n :

$H[A[i][0]] = \text{true}$

$\left. \begin{array}{l} \text{iterar pelos TOP 10\% seguidores} \\ \text{e salva eles no hash} \end{array} \right\}$

PARTICIONA POR ELEMENTO($A, 0, n, \text{topengajamento}, 3$) $\rightarrow$ todos depois do índice $\frac{9n}{10}$ são TOP 10% engajamento

define aux = 0

para $i = \frac{9n}{10}$ até n :

$\rightarrow$ iterar pelos top 10% engajamento

se $H[A[i][1]] \neq \text{true}$:

$\rightarrow$ se não é top 10% seguidores, então é emergente

emergentes[aux] = $A[i][1]$

$\rightarrow$ adiciona nos emergentes

aux++

return emergentes, aux $\rightarrow$ return emergentes e contagem deles.

Como trata particionar quanto com MOM são operações lineares, temos que a complexidade dessas partes será $O(n)$. Iterar pelos top 10% (tanto seguidores quanto engajamento) são operações em $O(n/10)$, já que dentro do heap apenas operações em tempo constante são realizadas.

Logo, a complexidade do algoritmo será $T(n) = O(n) + O(n) + O(n) + O(n) + O(n/10) + O(n/10)$

$= O(n)$

✓ 2.0

ENCONTRA MAIOR SUBLISTA (A, n)

define melhor par = $(-1, -1)$

define melhor soma = $-\infty$

- para $i = 0$ até $n-1$ $O(n)$

- para $j = i+1$ até n $O(n)$

soma atual = 0

- para $k = i$ até j $O(n)$

soma atual += $A[k]$

se soma atual > melhor soma:

melhor soma = soma atual

melhor par = (i, j)

retorna melhor par, melhor soma

três $O(n)$ sendo executados

um dentro do outro

$$= n \cdot n \cdot n = n^3$$

$$O(n^3)$$

✓ 0.5

O que o algoritmo faz é bem simples: para todo par (i, j) com $i < j$, testa a soma e vê se ela é melhor que a última melhor encontrada. É simples, porém a complexidade desse algoritmo: 3 loops em $O(n)$ não realizados um dentro do outro, resultando em complexidade $O(n^3)$

b) É possível otimizar sabendo a soma anterior e apenas modificando-a;

ENCONTRA MAIOR SUBLISTA (A, n)

define melhor par = $(-1, -1)$

define melhor soma = $-\infty$

define soma atual = 0

- para $i = 0$ até $n-1$:

soma atual = $A[i]$

- para $j = i+1$ até n :

soma atual += $A[j]$

se soma atual > melhor soma:

melhor soma = soma atual

melhor par = (i, j)

retorna melhor soma, melhor par

Caso as operações no segundo loop não em tempo constante, então vamos para $O(n^2)$

✓ 1.5

c) Sim, é possível.

ENCONTRA MAIOR SUBLISTA(A, n)

define melhor par = (-1, 1)

define melhor soma = $-\infty$

define soma atual = A[0]

define j = 0

para j = 1 até n:

soma atual += A[j]

se soma atual - A[j] > soma atual:

j++

soma atual = soma atual - A[j]

se soma atual > melhor soma

melhor soma = soma atual

melhor par = (i, j)

para i = 1 até n:

soma atual = A[i]

se soma atual > melhor soma:

melhor soma = soma atual

melhor par = (i, j)

retorna melhor par, melhor soma

$O(1)$

+

iterou uma vez e realizou apenas
operações em tempo constante

$O(n)$

+

iterou uma vez de i até n

e fez operações constantes

$O(n)$

=

$O(n)$

No lugar de avaliar todas as combinações de i e j possíveis, o algoritmo

avança j até o final da lista e avança para o próximo i e deve avançar i
e montá-lo onde está. Ao final, ele avança i até o final da array.

Deu certo, se não há outras combinações de i e j que já não sejam máximas,
pois não se avança i e não é necessário.

X 1.0